ENSy: Design, Implementation, and Application of an Interactive Embedding Navigation System for Large Language Models

Author: Alexandre da Silva Santos
Institution: Federal University of Maranhão
Program: B.Sc. in Science and Technology – Computer Engineering
Date: August 2026
Version: 2.0.0

Abstract

The internal representations of large language models, commonly referred to as word embeddings, encode rich semantic and syntactic relationships in high-dimensional vector spaces that resist straightforward human inspection. This work presents **ENSy (Embedding Navigation System)**, an interactive, browser-based computational environment designed to enable the navigation, exploration, and geometric analysis of high-dimensional embedding spaces extracted from three large language model architectures: Qwen 2.5 0.5B-Instruct (decoder-only, $d = 896$), Gemma 2B (decoder-only), and BERT base uncased (encoder-only). The system integrates a Python-based backend, built with FastAPI and Uvicorn, with a TypeScript and React frontend rendered using the Three.js WebGL engine. Four dimensionality reduction algorithms — UMAP, t-SNE, PCA, and Isomap — project embedding vectors extracted from each model's input embedding matrix onto a navigable three-dimensional Cartesian manifold. Projection quality is assessed through a co-ranking framework (Lee & Verleysen, 2009), computing Trustworthiness, Continuity, the behavioral indicator $B_{NX}(K)$, and the general preservation metric $Q_{NX}(K)$ over a shared lexical dataset derived from the Penn Treebank. A per-token decomposition $B_{NX,i}(K)$ drives a thermal-camera color overlay enabling local geometric distortion inspection. A central-reticle screen-space raycasting mechanism enables the user to select individual token representations and inspect their topological neighborhood through a three-layer color system distinguishing true, false, and missing neighbors. This document provides a comprehensive technical description of the system's architecture, algorithmic foundations, implementation choices, data flow, and user interaction model.

Keywords: word embeddings, embedding navigation, dimensionality reduction, co-ranking, trustworthiness, continuity, B_{NX}, Penn Treebank, UMAP, t-SNE, PCA, Isomap, cosine similarity, Three.js, FastAPI, Qwen 2.5, Gemma 2B, BERT, interpretability.

1. Introduction

Modern large language models (LLMs) rely on embedding matrices to map discrete tokens — linguistic units such as words, subwords, or characters — into continuous, dense vectors in a high-dimensional real-valued space $\mathbb{R}^d$. These embedding vectors serve as the primary substrate through which LLMs encode semantic similarity, syntactic relations, and contextual meaning. The geometric structure of these spaces is not arbitrary: tokens with related meanings tend to occupy proximal regions, forming clusters, sub-manifolds, and directional orientations that encode analogical relationships, such as:

$$\vec{v}_{\text{king}} - \vec{v}_{\text{man}} + \vec{v}_{\text{woman}} \approx \vec{v}_{\text{queen}}$$

Despite the theoretical richness of these spaces, their high dimensionality — ranging from 128 to 4096 dimensions in contemporary models — makes them inherently

opaque to direct human inspection. Researchers analyzing model behavior for the purposes of interpretability, bias detection, or curriculum learning must rely on mathematical abstractions and indirect proxies. This opacity represents a significant bottleneck in the field of explainable artificial intelligence (XAI), where understanding the internal geometry of a model is prerequisite to understanding its behavior.

Dimensionality reduction techniques such as Principal Component Analysis (PCA), t-distributed Stochastic Neighbor Embedding (t-SNE), Uniform Manifold Approximation and Projection (UMAP), and Isomap offer a bridge between the native high-dimensional space and lower-dimensional projections that preserve topological or statistical structure to varying degrees. However, these transformations introduce distortions whose nature and magnitude vary across algorithms and datasets. Existing tools for applying these techniques to language model embeddings are often limited to static 2D scatter plots, require specialized data science environments, or lack quantitative assessment of projection fidelity.

This work addresses these limitations by presenting **ENSy (Embedding Navigation System)**: a full-stack, interactive platform that renders large language model embedding spaces as navigable 3D manifolds in the browser and quantifies geometric distortions through the rank-based co-ranking framework of Lee & Verleysen (2009). ENSy empowers researchers to explore high-dimensional geometries via 6-DOF flight controls, inspect local neighborhoods using screen-space reticle raycasting, contrast four projection algorithms (UMAP, t-SNE, PCA, Isomap), assess projection quality through co-ranking metrics, and visualize local distortion contributions via a thermal camera overlay.

The contribution of this work is both theoretical and practical. It provides a concrete implementation of an XAI instrument for language model geometry, demonstrates the architectural patterns required to couple a machine learning inference pipeline with a real-time 3D rendering engine, and offers an accessible platform for pedagogical exploration of embedding spaces at the undergraduate and graduate levels.

1.1 Research Objectives

The research program guiding this work is articulated through five specific objectives:

O1 — Full-Stack Manifold Rendering. To design and implement an interactive web architecture capable of extracting language model embeddings from multiple large language model architectures and rendering them as navigable 3D Riemannian manifold approximations.

O2 — Multi-Algorithmic and Multi-Model Comparative Analysis. To integrate linear (PCA) and non-linear (t-SNE, UMAP, Isomap) dimensionality reduction engines behind a unified asynchronous API, enabling comparative inspection across four algorithms and three model architectures (Qwen 2.5 0.5B-Instruct, Gemma 2B, BERT base uncased), yielding 12 experimental configurations.

O3 — Quantitative Projection Quality Assessment. To implement the rank-based co-ranking framework (Lee & Verleysen, 2009) computing Trustworthiness $M_T(K)$, Continuity $M_C(K)$, the behavioral indicator $B_{NX}(K)$, and general preservation $Q_{NX}(K)$ as curves over $K \in \{5, 10, 15, 20\}$, complemented by a Silhouette score with the reference linguistic partition.

O4 — Topological Neighborhood Inspection. To implement a screen-space proximity selection mechanism that dynamically exposes local topological structures through a three-layer color system (reference partition, thermal camera $B_{NX,i}(K)$, neighborhood probing $T_K/F_K/M_K$).

O5 — Interpretability and Methodological Rigor. To demonstrate the system's practical efficacy as an explainable AI tool while maintaining a reproducible experimental protocol with a fixed dataset derived from the Penn Treebank, congelated hyperparameter tables, and seed-fixed stochastic algorithms.

1.2 Document Organization

This document is organized as follows. Section 2 provides the theoretical background on word embeddings, dimensionality reduction algorithms, and the co-ranking quality assessment framework. Section 3 describes the system architecture in full, encompassing the technology stack, component model, and data flow. Section 4 details the backend implementation, including the embedding pipeline, projection endpoints, and metrics computation. Section 5 describes the frontend implementation, with emphasis on the Three.js rendering engine, camera control model, raycasting selection mechanism, and visualization layers. Section 6 discusses user interaction design and interface layout. Section 7 presents the data model. Section 8 discusses limitations and directions for future work. Section 9 concludes.

2. Theoretical Background

2.1 Word Embeddings and the Distributional Hypothesis

The mathematical foundation of word embeddings rests on the **distributional hypothesis**, which asserts that words appearing in similar linguistic contexts tend to share semantic properties. Formally, given a vocabulary $\mathcal{V}$ of $|\mathcal{V}|$ tokens, an embedding model defines a mapping:

$$f : \mathcal{V} \to \mathbb{R}^d$$

mapping each token $w_i \in \mathcal{V}$ to a dense vector $\vec{v}_i \in \mathbb{R}^d$, where $d \ll |\mathcal{V}|$.

ENSy extracts exclusively the **input embedding matrix** of each model — `get_input_embeddings().weight` for Qwen 2.5 and Gemma 2B (decoder-only), and `word_embeddings.weight` for BERT (encoder-only). This representation is directly comparable across the three architectures because it corresponds to the token-to-vector mapping applied before any contextual attention layer, making it architecture-agnostic at the point of extraction. Extracting hidden states from intermediate transformer layers would require decisions about layer depth, context, and pooling strategy that would compromise cross-architecture comparability.

The three models differ not only in architecture (decoder-only vs. encoder-only) but also in embedding dimensionality, vocabulary size, tokenizer, pre-training objective, corpus, and training recipe. ENSy's experimental design therefore permits observing whether distortion patterns of dimensionality reduction algorithms are consistent or variable across different models and architectures, without attributing causality exclusively to any single factor.

Model	Architecture	Embedding dimension d	Vocab. size $\lvert\mathcal{V}\rvert$
Qwen 2.5 0.5B-Instruct	Decoder-only (Transformer)	896	151,936
Gemma 2B	Decoder-only (Transformer)	2,048	256,000
BERT base uncased	Encoder-only (Transformer)	768	30,522

2.2 Cosine Similarity

The **cosine similarity** between two embedding vectors $\vec{u}, \vec{v} \in \mathbb{R}^d$ is defined as:

$$\text{sim}_{\cos}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|\|\vec{v}\|} = \frac{\sum_{k=1}^{d} u_k v_k}{\sqrt{\sum_{k=1}^{d} u_k^2}\sqrt{\sum_{k=1}^{d} v_k^2}}$$

The cosine similarity ranges in the interval $[-1, 1]$, where a value of $+1$ indicates identical orientation, 0 indicates orthogonality, and -1 indicates opposite orientation. This metric is particularly well-suited for embedding spaces because it is invariant to vector magnitude, focusing exclusively on the angular relationship between vectors.

All embedding vectors are subjected to **mandatory L_2 normalization** after extraction and before any metric computation. This normalization renders cosine similarity equivalent to Euclidean distance on the unit hypersphere ($\|\hat{u} - \hat{v}\|^2 = 2(1 - \cos\theta)$), ensuring consistent distance semantics throughout the pipeline. Normalization does not eliminate anisotropy in the embedding space; it solely establishes geometric equivalence between cosine and Euclidean metrics for the purpose of distance computation.

In ENSy, the high-dimensional neighborhood structure used by all co-ranking metrics is computed using the cosine distance matrix $D_X = D_{\cos}(X)$. Projection algorithms that accept a metric parameter (t-SNE, UMAP, Isomap) are explicitly configured with `metric="cosine"` to ensure consistency between the neighborhood graph construction and the metric used for quality assessment.

2.3 Dimensionality Reduction

Since native embedding dimensions ($d = 768$ to $d = 2048$) cannot be directly visualized, dimensionality reduction to $k = 3$ is required for the interactive 3D manifold. ENSy implements four distinct algorithms, each offering different preservation guarantees and structural emphases. All algorithm parameters are fixed prior to any data collection as specified in the congelated parameter table (Section 4.5).

2.3.1 Principal Component Analysis (PCA)

PCA is a classical linear method that projects data onto a lower-dimensional subspace defined by the directions of maximum variance in the original data. Given the centered data matrix $X_c = X - \mu \in \mathbb{R}^{N \times d}$, PCA is implemented via direct Singular Value Decomposition:

$$X_c = U\Sigma V^\top, \qquad Y = X_c V_{1:3}$$

selecting the $k = 3$ right singular vectors corresponding to the three largest singular values as the projection axes. Constructing the explicit covariance matrix $C = \frac{1}{N} X^\top X$ is avoided for numerical stability. PCA is **globally structure-preserving**: the first principal component captures the maximum variance, and

subsequent components capture progressively less variance. However, PCA is a linear transformation and cannot capture nonlinear manifold structure.

A **PCA-NC (non-centered)** variant, which omits the centering step $X_c = X - \mu$, is available as an explicit ablation mode. PCA-NC is never reported as "PCA" without qualification; results from this ablation are documented and reported separately from the 12 principal experimental configurations.

2.3.2 t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE is a nonlinear probabilistic method that preserves **local neighborhood structure** in the projection. t-SNE constructs a probability distribution P over pairs of points in the high-dimensional space, where the probability p_{ij} reflects how likely points i and j are to be neighbors under a Gaussian kernel with the cosine distance metric. In the low-dimensional space, a similar distribution Q is defined using a Student's t-distribution with one degree of freedom. The projection is optimized by minimizing the Kullback-Leibler divergence between P and Q:

$$\mathcal{L}_{\text{t-SNE}} = \mathrm{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

The **perplexity** is fixed at 5 for all experimental configurations, adopted due to the small sample size of the experiment. The high-dimensional neighborhood graph is constructed using $D_X = D_{\cos}(X)$; the evaluation and visualization operate on $D_Y = D_{\text{euclidean}}(Y)$ in the projected space $\mathbb{R}^3$. A key limitation of t-SNE is that it does not preserve global distances: cluster separations in the t-SNE projection are not directly interpretable as inter-cluster distances in the original space.

2.3.3 Uniform Manifold Approximation and Projection (UMAP)

UMAP is a manifold learning technique grounded in Riemannian geometry and algebraic topology. UMAP constructs a weighted graph representation of the high-dimensional data, approximating the manifold on which the data lies, then optimizes a low-dimensional layout by minimizing the cross-entropy between the high- and low-dimensional fuzzy topological representations. UMAP preserves both **local and global structure** better than t-SNE and is significantly faster for large datasets.

ENSy applies UMAP with the following congelated parameterization (library defaults):

$$\begin{aligned}\mathrm{UMAP}\big(&n_{\text{components}} = 3,\\ &n_{\text{neighbors}} = 15,\ \text{min_dist} = 0.1,\\ &\text{metric} = \text{"cosine"},\ \text{random_state} = 42\big)\end{aligned}$$

The library default values for `n_neighbors` and `min_dist` are adopted deliberately: using documented defaults rather than values selected to produce visually appealing projections protects against the criticism that hyperparameters were adjusted post-hoc to favor a particular result.

2.3.4 Isomap

Isomap is a nonlinear method that extends classical multidimensional scaling (MDS) to preserve **geodesic distances** on the data manifold. Isomap first constructs a neighborhood graph with k nearest neighbors using the cosine metric, then estimates geodesic distances as shortest path lengths in this graph using Dijkstra's algorithm. Finally, classical MDS is applied to the geodesic distance matrix to produce the low-dimensional embedding. Isomap is particularly effective for data lying on a smoothly curved low-dimensional manifold.

In ENSy, Isomap uses $n_{\text{neighbors}} = 15$ and `metric="cosine"` for graph construction. The neighborhood scale of 15 is adopted to ensure approximate comparability with UMAP's neighborhood graph, not as a canonical value for the algorithm.

2.4 Co-ranking Framework and Projection Quality Metrics

ENSy adopts the rank-based co-ranking framework of Lee & Verleysen (2009) as the primary quantitative instrument for assessing the geometric fidelity of each dimensionality reduction, following the implementation order: Q matrix → Trustworthiness/Continuity → B_{NX} → Q_{NX} → Silhouette.

2.4.1 Co-ranking Matrix

Given N points, let r_{ij} denote the rank of point j with respect to point i in the high-dimensional space (under $D_X = D_{\cos}$), and $\hat{r}_{ij}$ the corresponding rank in the projected space (under $D_Y = D_{\text{euclidean}}$). The co-ranking matrix is defined as:

$$\begin{aligned}Q &= [q_{kl}]_{1 \le k,l \le N-1},\\ q_{kl} &= \left|\{(i,j) : r_{ij} = k \text{ and } \hat{r}_{ij} = l\}\right|\end{aligned}$$

The computational cost is $O(N^2 \log N)$ (two sorting operations). The matrix Q is partitioned into four blocks of size K: UL_K (upper-left), UR_K (upper-right), LL_K (lower-left), LR_K (lower-right). The block UL_K is further divided into diagonal D_K, lower triangle

$LT_K = \{(k,l) : 1 < k \le K,\ l < k\}$, and upper triangle $UT_K = \{(k,l) : 1 \le k < K,\ l > k\}$.

Intrusions and Extrusions. An **intrusion** occurs when $\hat{r}_{ij} \le K < r_{ij}$: point j appears as a false neighbor of i in the projected K-neighborhood without being a genuine neighbor in the high-dimensional space. An **extrusion** occurs when $r_{ij} \le K < \hat{r}_{ij}$: a genuine high-dimensional neighbor is displaced outside the projected K-neighborhood. Mild (light) intrusions satisfy $\hat{r}_{ij} < r_{ij} \le K$; mild extrusions satisfy $r_{ij} < \hat{r}_{ij} \le K$.

2.4.2 Trustworthiness and Continuity

Trustworthiness $M_T(K)$ quantifies the absence of false neighbors, while Continuity $M_C(K)$ quantifies the preservation of original neighbors:

$$M_T(K) = 1 - \frac{2}{G_K} \sum_{(k,l)\in LL_K} (k - K)\, q_{kl}$$

$$M_C(K) = 1 - \frac{2}{G_K} \sum_{(k,l)\in UR_K} (l - K)\, q_{kl}$$

$$G_K = \begin{cases} NK(2N - 3K - 1) & \text{if } K < N/2 \\ N(N-K)(N-K-1) & \text{if } K \ge N/2 \end{cases}$$

Trustworthiness is penalized by severe K-intrusions (distant vectors becoming false neighbors); Continuity is penalized by severe K-extrusions (genuine neighbors displaced to far-away projected positions).

2.4.3 Behavioral Indicator $B_{NX}(K)$

The mild intrusion and extrusion fractions are defined as (Lee & Verleysen, 2009, Eq. 22–23):

$$U_N(K) = \frac{1}{KN} \sum_{(k,l)\in LT_K} q_{kl}$$

(fraction of mild K-intrusions)

$$U_X(K) = \frac{1}{KN} \sum_{(k,l)\in UT_K} q_{kl}$$

(fraction of mild K-extrusions)

$$\boxed{B_{NX}(K) = U_X(K) - U_N(K)}$$

A positive value indicates **intrusive** behavior (crowding, as characteristically produced by t-SNE in sparse regions); a negative value indicates **extrusive** behavior (tearing, as produced by Isomap under component collapse); a value near zero indicates approximately balanced behavior. The apparently counterintuitive sign convention — where intrusive behavior yields $B_{NX} > 0$ from a formula involving U_X (mild extrusions) — is explained structurally: the number of severe K-intrusions always equals the number of severe K-extrusions (a property of every row and column of Q summing to exactly N), so a single severe intrusion produces multiple mild extrusions proportional to the intrusion's severity. The magnitude of U_X therefore indirectly measures the severity of intrusions, and vice versa (Lee & Verleysen, 2009, Section 4, Fig. 2).

2.4.4 General Preservation $Q_{NX}(K)$

The overall preservation of K-ary neighborhoods against both intrusive and extrusive distortions is quantified by $Q_{NX}(K)$:

$$\begin{aligned} Q_{NX}(K) &= U_P(K) + U_N(K) + U_X(K) \\ &= Q_{LC}(K) + \frac{K}{N-1} \end{aligned}$$

where $U_P(K) = \frac{1}{KN} \sum_{(k,l)\in D_K} q_{kl}$ is the fraction of pairs maintaining exactly the same rank in both neighborhoods, and $Q_{LC}(K)$ is the Local Continuity Meta-Criterion (LCMC). $Q_{NX}(K)$ decreases only with severe K-intrusions and K-extrusions.

2.4.5 Per-Token Behavioral Indicator $B_{NX,i}(K)$

The thermal camera visualization requires a per-token local score. Each entry q_{kl} decomposes as $q_{kl} = \sum_{i=1}^{N} q_{kl}^{(i)}$, where $q_{kl}^{(i)}$ counts, for a fixed point i, the j such that $r_{ij} = k$ and $\hat{r}_{ij} = l$. Applying the same triangular masks of Section 2.4.3 to $q_{kl}^{(i)}$:

$$U_{N,i}(K) = \frac{1}{K} \sum_{(k,l)\in LT_K} q_{kl}^{(i)}$$

$$U_{X,i}(K) = \frac{1}{K} \sum_{(k,l)\in UT_K} q_{kl}^{(i)}$$

$$\boxed{B_{NX,i}(K) = U_{X,i}(K) - U_{N,i}(K)}$$

This definition satisfies the **consistency property** (verified as an automated test before thermal camera deployment):

$$B_{NX}(K) = \frac{1}{N} \sum_{i=1}^{N} B_{NX,i}(K)$$

The local score ranges approximately in $[-1, 1]$, enabling a fixed bipolar scale for the thermal camera that does not require per-frame or per-projection dynamic normalization.

2.4.6 Silhouette Score

For each projected token i, the Silhouette coefficient $s(i)$ evaluates cluster tightness and category separation:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

where $a(i)$ is the mean intra-category distance and $b(i)$ is the mean nearest-category distance (Rousseeuw, 1987), computed over the projected coordinates $Y_i \in \mathbb{R}^3$ using

`content_noun` / `content_verb` / `function_word` as the reference linguistic partition. The Silhouette provides a complementary measure of inter-category separation in the projected space; it measures separation, not the cause (crowding, tearing, etc.). Class-Conditional Centering is never applied before this computation, as using class labels to center prior to an unsupervised projection algorithm would artificially inflate inter-category separation and invalidate the metric.

2.5 The Semantic Manifold

The concept of "manifold" in ENSy's nomenclature refers to the mathematical concept of a topological manifold: a space that locally resembles Euclidean space but may possess complex global geometry. The embedding vectors of a language model are hypothesized to lie near a low-dimensional manifold embedded in the high-dimensional vector space $\mathbb{R}^d$. The dimensionality reduction algorithms described above are manifestations of the **manifold hypothesis**: the assumption that high-dimensional data of natural origin concentrates near a low-dimensional manifold, and that this manifold structure encodes meaningful relationships.

3. System Architecture

ENSy follows a two-tier client-server architecture, separating the computationally intensive embedding extraction, dimensionality reduction, and co-ranking metric computation from the interactive rendering and user interface.

3.1 Technology Stack

ENSy leverages modern web and scientific computing frameworks structured across four primary layers:

Frontend and Graphics Engine. React (v19.0.1) and TypeScript (~v5.8.2) orchestrate component state and static type verification, while Three.js (v0.184.0) manages the WebGL 3D scene graph, shader routines, and canvas rendering. Vite (v6.2.3) serves as the high-throughput development bundler, Tailwind CSS (v4.1.14) and Motion (v12.23.24) govern responsive layout and micro-animations, MathJax (v3) provides client-side LaTeX equation rendering, and Lucide React (v0.546.0) delivers vector iconography.

Backend and Asynchronous Serving. High-throughput REST API serving is provided by FastAPI (v0.115.8) running on the asynchronous Uvicorn (v0.34.0) ASGI server.

Machine Learning and Reduction Pipeline. Token embedding extraction is performed on Qwen 2.5 0.5B-Instruct, Gemma 2B, and BERT base uncased using PyTorch (v2.6.0) and HuggingFace Transformers (v4.49.0). Dimensionality reduction algorithms (PCA, t-SNE, Isomap, UMAP) are executed via scikit-learn (v1.6.1) and UMAP (v0.5.7), supported by vectorized NumPy (v2.1.3) linear algebra.

Verification and Quality Assurance. Automated endpoint validation and visual regression testing are conducted using Pytest (v8.3.5) and headless Playwright (v1.48.0). Synthetic co-ranking test cases (Section 4.6) provide pre-deployment validation of all metric implementations.

3.2 Data Flow

The system's data flow proceeds through the following stages:

Stage 1 — Dataset Construction. Prior to any embedding extraction or metric computation, the lexical dataset is built from the Penn Treebank (Section 4.2). The resulting `dataset.json` is versioned and its SHA-256 hash is recorded. This file is congelated and serves as a fixed experimental input.

Stage 2 — Application Initialization. Upon server startup (`app.py`), the dependency installer verifies and installs missing Python packages. The cached embedding matrices for all three models are loaded from memory if available; otherwise, a daemon thread is launched to load each model sequentially.

Stage 3 — Model Loading (Background). The background thread loads each model's tokenizer and weights via the HuggingFace Transformers library. The input embedding matrix E is extracted via `get_input_embeddings().weight` (Qwen/Gemma) or `bert.embeddings.word_embeddings.weight` (BERT), cast from `float16` to `float32`, and L_2-normalized. Only lexical items satisfying the single-token criterion for that model's tokenizer are retained. Upon successful loading, the global `active_vectors` pointer for that model is atomically updated.

Stage 4 — API Request. The React frontend issues an HTTP GET request to `/api/projection/{model_id}/{engine_type}` whenever the user changes the model or projection algorithm.

Stage 5 — Projection and Metric Computation. The backend applies the requested dimensionality reduction algorithm to `active_vectors` for the selected model, producing 3D coordinates for each token. It then computes the full co-ranking matrix Q and derives all

quality metrics: $M_T(K)$, $M_C(K)$, $B_{NX}(K)$, $Q_{NX}(K)$, $B_{NX,i}(K)$ for $K \in \{5, 10, 15, 20\}$, and the Silhouette score.

Stage 6 — JSON Response. The backend returns a JSON object conforming to the `SemanticSpace` interface. The frontend's `App.tsx` stores this in the `semanticSpace` state variable.

Stage 7 — 3D Scene Update. `SpaceVisualizer` detects the updated data and calls `syncProjectionDataPrivate()`, which linearly interpolates each token's 3D point towards its new projected position over subsequent animation frames. Token colors are updated by rewriting the `BufferGeometry` vertex color attribute.

Stage 8 — Offline Fallback. If the API request fails, `App.tsx` catches the exception and loads the static `semanticSpaceData` from `src/data/semanticSpace.ts`, ensuring the application remains functional.

4. Backend Implementation

4.1 Server Entrypoint and Dependency Management

The backend server (`app.py`) begins with a self-bootstrapping dependency management routine (`install_deps()`). Before importing any scientific computing library, the function iterates over a required package list (`fastapi`, `uvicorn`, `transformers`, `torch`, `scikit-learn`, `umap-learn`) and attempts an import for each. Any package that fails the import test is installed via `subprocess.check_call([sys.executable, "-m", "pip", "install", ...])`. This strategy eliminates the need for a pre-configured environment.

4.2 Lexical Dataset — Penn Treebank Pipeline

ENSy's experimental vocabulary is a **lexical dataset derived from the Penn Treebank**, not a fixed hardcoded word list. The dataset is built by a 10-step curation pipeline that produces a frozen, reproducible lexical input prior to any embedding extraction or metric computation:

1. **Corpus Extraction:** Extract token surface forms from the Penn Treebank corpus (PTB tokens $\rightarrow$ lexical types).
2. **Text Normalization:** Apply case normalization (lowercasing) and punctuation/symbol stripping.
3. **Linguistic Categorization:** Assign each candidate form a reference linguistic category: `content_noun`, `content_verb`, or `function_word`.
4. **Homograph Disambiguation:** Resolve homographs appearing under multiple categories using a fixed corpus-frequency rule: the category of highest frequency in the corpus is assigned. Forms whose category cannot be determined unambiguously by this rule are **excluded** from the dataset.
5. **Qwen Tokenization:** Perform independent tokenization using the Qwen 2.5 tokenizer.
6. **Gemma Tokenization:** Perform independent tokenization using the Gemma 2B tokenizer.
7. **BERT Tokenization:** Perform independent tokenization using the BERT base uncased tokenizer.
8. **Single-Token Intersection Filtering:** Retain only candidate forms that map to **exactly one token ID** across all three tokenizers and correspond to valid rows of each model's input embedding matrix.
9. **Strict Deduplication:** Ensure each normalized surface form appears at most once in the dataset.
10. **Category Balancing and Hash Freezing:** Balance the representation across the three reference categories and record the resulting N items as `dataset.json` accompanied by its SHA-256 integrity hash.

This 10-step curation constitutes Phase 1 of the system's **21-step operational sequence**: Steps 1–10 produce and freeze `dataset.json`; Steps 11–13 extract input embeddings across the three models, verify native dimensions ($d = 896, 2048, 768$), and apply mandatory L_2 normalization; Steps 14–16 compute the cosine distance matrix D_X, construct the co-ranking matrix Q, and evaluate rank-based quality metrics; Steps 17–20 execute the four dimensionality reduction engines (PCA, t-SNE, UMAP, Isomap); and Step 21 executes and consolidates the 12 experimental configurations.

The final token count N is reported as the empirical output of this pipeline — it is not artificially inflated to any predetermined target. The `dataset.json` file and its SHA-256 hash are versioned in the repository as immutable experimental inputs.

Taxonomy note. The partition `content_noun` / `content_verb` / `function_word` is designated the **reference linguistic partition** — not "grammatical classes" or "stopwords." The label assigned to each form in the dataset represents the chosen category in the curated set, not an inference from model representations.

4.3 Embedding Cache

Upon model loading, embedding vectors for the N dataset tokens are extracted, cast to `float32`, and L_2-normalized. The normalized matrices are stored in an in-memory cache keyed by:

```
cache_key = (
    model_id,
    dataset_hash,
    extraction_config,
    preprocessing_config,
)
```

This cache guarantees zero recomputation when the user switches projection algorithms for the same model, while ensuring that any change to the dataset or extraction configuration triggers a full recomputation. No disk-based cache is used; reproducibility is guaranteed by the `dataset_hash` and fixed `random_state=42` for stochastic algorithms.

4.4 Asynchronous Model Loading

The model loading procedure is executed asynchronously via a Python daemon thread spawned upon server startup (see Appendix D, Listing D.2).

The background initialization sequence executes three sequential stages per model:

Tokenizer and Architecture Instantiation. The pipeline instantiates the HuggingFace `AutoTokenizer` and `AutoModelForCausalLM` (Qwen, Gemma) or `AutoModel` (BERT) using half-precision (`float16`) to minimize memory footprint.

Embedding Matrix Extraction. The input embedding weights E are decoupled from the PyTorch computation graph via `get_input_embeddings().weight` (Qwen/Gemma) or `bert.embeddings.word_embeddings.weight` (BERT), cast to `float32` NumPy arrays, and L_2-normalized.

Single-Token Validation. Each candidate dataset term is evaluated with the model's tokenizer. Forms that decompose into multiple subwords are excluded, consistent with the dataset construction criterion of Section 4.2.

Atomic Pointer Replacement. Following validation, the global references `active_words`, `active_vectors`, and model metadata are updated simultaneously. Python's Global Interpreter Lock (GIL) ensures atomic reference reassignment.

Note on Gemma 2B. Gemma 2B is a gated model on HuggingFace Hub, requiring license acceptance and token-based authentication before programmatic download. Credentials must be configured before server startup.

4.5 Projection Endpoint and Congelated Parameter Table

The core API endpoint is `GET /api/projection/{model_id}/{engine_type}`, accepting `qwen`, `gemma`, o `bert` as `model_id` and `umap`, `pca`, `tsne`, or `isomap` as `engine_type`.

All algorithm parameters are fixed prior to data collection and are not modified during experimentation:

Algorithm	Parameter	Value
PCA	n_components	3
PCA	centering	yes (ablation: PCA-NC, non-centered)
t-SNE	perplexity	5 (fixed; adopted for small sample size)
t-SNE	random_state	42
t-SNE	metric (high-dim)	cosine
UMAP	n_neighbors	15 (library default)
UMAP	min_dist	0.1 (library default)
UMAP	metric	cosine
UMAP	random_state	42
Isomap	n_neighbors	15 (comparable scale to UMAP)
Isomap	metric	cosine

For stochastic algorithms (t-SNE, UMAP): a fixed seed guarantees reproducibility of the reported result but does not characterize stability. Sensitivity analysis with multiple seeds is a separate analysis, never mixed with the 12 principal experimental results.

The endpoint returns 3D projected coordinates and all co-ranking metrics for the requested (model, algorithm) configuration.

4.6 Co-ranking Metrics Pipeline

Following projection, the backend computes the full co-ranking metric suite for $K \in \{5, 10, 15, 20\}$.

Co-ranking matrix computation ($O(N^2 \log N)$): rank matrices R and $\hat{R}$ are built by sorting D_X and D_Y

independently for each point; q_{kl} is accumulated by iterating over all pairs.

Trustworthiness and Continuity ($M_T(K)$, $M_C(K)$): computed from the LL_K and UR_K blocks of Q per the formulas of Section 2.4.2.

Behavioral indicator $B_{NX}(K) = U_X(K) - U_N(K)$: computed from the LT_K and UT_K triangular blocks of Q per the source definition (Lee & Verleysen, 2009, Eq. 22–23).

Per-token decomposition $B_{NX,i}(K)$: computed by decomposing $q_{kl}^{(i)}$ per point before aggregation. The consistency property $B_{NX}(K) = \frac{1}{N}\sum_i B_{NX,i}(K)$ is verified as an automated test (tolerance $\epsilon < 10^{-6}$) before any thermal camera deployment.

Silhouette score: computed over the projected $\mathbb{R}^3$ coordinates using the reference linguistic partition, without any Class-Conditional Centering applied beforehand.

Results for all 12 configurations (3 models × 4 algorithms) are versioned alongside the `dataset_hash` and the library version manifest.

4.7 Static File Serving and SPA Routing

In production mode, FastAPI mounts the `dist/` directory and serves the React SPA. The fallback route `GET /{rest_of_path:path}` returns `index.html` for any non-API path, enabling client-side React navigation.

5. Frontend Implementation

5.1 Component Architecture

The frontend architecture is partitioned across four specialized React components coordinated by the root `App.tsx`. The root orchestrator maintains global application state (`selectedModel`, `selectedEngine`, `selectedNode`, `semanticSpace`, `activeColorLayer`), handles HTTP communication with the backend API, and manages fallback routing. `SidebarLeft.tsx` hosts model and algorithm selectors, co-ranking metric panels, and server connectivity monitors. `SpaceVisualizer.tsx` encapsulates the Three.js WebGL canvas, 6-DOF camera mechanics, HUD overlays, and the three-layer color system. `SidebarRight.tsx` inspects node coordinates, presents co-ranking metrics for the selected token, and provides CSV exports. `BottomHUD.tsx` renders non-intrusive navigation indicators.

All inter-component communication occurs through unidirectional props and callbacks, preserving deterministic state propagation via `useState` and `useCallback` hooks.

5.1.1 State Persistence

Application settings are debounced and serialized to `localStorage` at one-second intervals to prevent frame drops during the 60 FPS animation loop. Persisted attributes include the active model (`modelId`), reduction engine (`projectionType`), active color layer (`colorLayer`), octant spatial filter (`currentFilter`), camera projection geometry (`cameraMode`), field of view (`fov`), coordinate axis visibility flags (`showXAxis`, `showYAxis`, `showZAxis`), reference grid planes, mouse sensitivity coefficients, and 6-DOF camera coordinates.

5.2 Three.js Scene Construction

The `SpaceVisualizer` component initializes the Three.js rendering pipeline within an empty-dependency `useEffect` hook, isolating the WebGL scene graph from React reconciliation cycles.

5.2.1 Scene Graph

The Three.js scene graph consists of a primary `THREE.Scene` configured with exponential atmospheric fog ($\text{FogExp2}(0xfcfcfc, 0.003)$), an ambient illumination source (`THREE.AmbientLight(0xffffff, 1)`), a composite `gridGroup` containing reference planes, axis arrows, and tick markers, and a single `THREE.Points` object backed by a `BufferGeometry` containing all token positions and vertex colors.

5.2.2 Dual Camera System

Three distinct camera modes require both a perspective and an orthographic camera:

Perspective mode (`THREE.PerspectiveCamera`): standard projection model; FOV is user-configurable from 10° to 120°.

Parallel (Orthographic) mode (`THREE.OrthographicCamera`): orthographic projection for engineering views. Bounds are dynamically synchronized to the perspective camera's current frustum:

$$\text{halfHeight} = \|\vec{p}_{\text{cam}}\| \cdot \tan\left(\frac{\text{FOV}}{2} \cdot \frac{\pi}{180}\right)$$
$$\text{halfWidth} = \text{halfHeight} \cdot \text{aspect}$$

Two-point perspective mode: approximation of architectural two-point perspective, implemented by injecting a vertical shear into the projection matrix: `persp.projectionMatrix.elements[9] = Math.tan(persp.rotation.x)`.

5.3 Camera Controller: 6-DOF Flight Mechanics

ENSy implements a **six-degrees-of-freedom (6-DOF) first-person flight control scheme** using the browser's Pointer Lock API. When the user clicks the viewport, `requestPointerLock()` captures the mouse cursor.

Mouse movement updates camera orientation in `YXZ` Euler rotation order:

$$\begin{aligned}\theta_y &\mathrel{-}= \Delta x_{\text{mouse}} \cdot \lambda \\ \theta_x &\mathrel{-}= \Delta y_{\text{mouse}} \cdot \lambda \\ \theta_x &= \text{clamp}\left(\theta_x, -\frac{\pi}{2} + 0.05, \frac{\pi}{2} - 0.05\right)\end{aligned}$$

where $\lambda = 0.0015$ is the default sensitivity and the pitch clamp prevents gimbal lock. Mouse movement is clamped to $[-60, 60]$ pixels per event.

Keyboard translation is computed each frame with time-delta scaling:

$$\begin{aligned}\vec{p}_{\text{cam}} &\mathrel{+}= \vec{d}_{\text{forward}} \cdot v \cdot \Delta t \quad \text{(W key)} \\ \vec{p}_{\text{cam}} &\mathrel{-}= \vec{d}_{\text{forward}} \cdot v \cdot \Delta t \quad \text{(S key)} \\ \vec{p}_{\text{cam}} &\mathrel{\pm}= \vec{d}_{\text{right}} \cdot v \cdot \Delta t \quad \text{(A/D keys)} \\ p_{y,\text{cam}} &\mathrel{\pm}= v \cdot \Delta t \quad \text{(R/F keys)}\end{aligned}$$

where $v = 14\ \text{u/s}$ and Δt is capped at $33\ \text{ms}$. The forward direction is projected onto the XZ plane to ensure horizontal movement regardless of pitch.

5.4 Camera Preset Transitions (LERP/SLERP)

Camera preset transitions use a **damped interpolation** scheme per-frame:

Position LERP (Linear Interpolation):

$$\begin{aligned}\vec{p}_{t+1} &= \vec{p}_t + 0.06 \cdot (\vec{p}_{\text{target}} - \vec{p}_t) \\ q_{t+1} &= \text{SLERP}(q_t, q_{\text{target}}, 0.06)\end{aligned}$$

Quaternion-based SLERP is used for rotation to prevent gimbal lock. Interpolation terminates when position distance < 0.01 and rotation angle < 0.01 rad.

Preset camera transitions align the viewport to canonical orthogonal and perspective projections: **Front** view targets position $(0, 0, 45)$ with rotation $(0, 0, 0)$; **Top** view targets $(0, 45, 0)$ with rotation $(-\pi/2, 0, 0)$; **Side** view targets $(45, 0, 0)$ with rotation $(0, \pi/2, 0)$; and **Isometric** view targets $(24, 18, 24)$ with pitch $\arctan(-18/\sqrt{1152})$ and yaw $\pi/4$.

5.5 Token Rendering and Label Overlay System

All N token positions are stored as vertices in a single `THREE.BufferGeometry`, with colors stored as a vertex attribute array (`geometry.setAttribute('color', new THREE.BufferAttribute(colorArray, 3))`), and the material configured with `vertexColors: true`. This consolidation produces a single draw call for the entire point cloud regardless of N, compared to N individual draw calls in a per-token object approach.

Token labels are rendered as canvas sprites (`THREE.Sprite` + `CanvasTexture`), where each label is drawn as text into an offscreen `<canvas>` and uploaded as a GPU texture billboarded toward the camera. This approach eliminates the per-frame DOM update cost of the CSS2DRenderer pattern (N `<div>` elements repositioned via JavaScript every frame), at the cost of texture memory and draw calls that scale significantly better than DOM manipulation for the expected N range (150–200 tokens).

Visual clarity is maintained through dynamic depth sorting, where $\text{z-index} = \lfloor 1000 - d \rfloor$ guarantees that foreground labels correctly occlude background points. Label transparency is governed by a distance attenuation function:

$$\text{opacity} = \max\left(0, \min\left(1, 1 - \frac{d - d_{\text{thresh}}}{90}\right)\right)$$

5.6 Three-Layer Color System

Token colors are managed through three distinct visual layers with a strict precedence rule: **Probing (Layer 3) > Thermal Camera (Layer 2) > Reference Partition (Layer 1)**. Switching layers is implemented by rewriting the vertex color array and marking `colorAttribute.needsUpdate = true` — an $O(N)$ operation, not a material swap per token.

Layer 1 — Reference Linguistic Partition (default). Each token is colored by its dataset category:

Category	Color
`content_noun`	Blue
`content_verb`	Orange
`function_word`	Teal

Layer 2 — Thermal Camera / $B_{NX,i}(K)$ (global toggle F2). The per-token behavioral indicator $B_{NX,i}(K)$ (Section 2.4.5) is mapped to a fixed bipolar scale $[-1, 1]$:

Range of $B_{NX,i}(K)$	Color	Interpretation
< 0	Blue/Cyan	Extrusive behavior (tearing)
≈ 0	Light grey (never green)	High fidelity / neutral
> 0	Red/Orange	Intrusive behavior (crowding)

Grey is reserved for the neutral point; green is reserved exclusively for Layer 3. The scale is global and fixed — it does not normalize dynamically per frame or per projection. Two algorithms with the same local value always receive the same color, enabling direct comparison.

Layer 3 — Neighborhood Probing (active only on the reticle-selected token, F3).

Color	Category	Definition
Green	True Neighbors (T_K)	Genuine high-dim neighbors preserved in projection
Red	False Neighbors (F_K)	False neighbors introduced by projection
Translucent blue	Missing Neighbors (M_K)	Genuine neighbors displaced outside neighborhood

The thermal camera label in the UI reads: **"Thermal Camera — local contribution to $B_{NX}(K)$"**, not "local distortion," to reflect the precise semantics of $B_{NX,i}(K)$ (Section 2.4.5).

5.7 Node Selection by Central Reticle

ENSy implements a screen-space reticle selection mechanism evaluated on every animation frame. The viewport center is established as $(c_x, c_y) = (W/2, H/2)$. For each candidate point $\vec{p}_i$ verified to lie in front of the camera plane ($(\vec{p}_i - \vec{p}_{\text{cam}}) \cdot \hat{d}_{\text{cam}} > 0$), the 2D Euclidean screen distance is computed as $d_i = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2}$. The token p^* that minimizes d_i within an activation radius of 25 px is selected, triggering the `onNodeHovered` state transition, highlighting the corresponding label with the `.highlighted-acad` CSS class, activating Layer 3 probing visualization for that token, and populating the Right Sidebar with the token's co-ranking metrics.

5.8 Token Position Interpolation on Projection Switch

When switching projection algorithms or models, new coordinates arrive from the API asynchronously. The `syncProjectionDataPrivate()` function updates each token's `targetPosition`, and the animation loop smoothly interpolates token coordinates via $\vec{p}_{t+1} = \vec{p}_t + 0.04 \cdot (\vec{p}_{\text{target}} - \vec{p}_t)$. This LERP factor produces a transition of approximately $50–100$ frames ($0.8–1.7$ s at 60 FPS), allowing the user to observe the topological reorganization of the embedding space as the algorithm changes.

5.9 Adaptive Grid Resolution

Grid tick density dynamically adapts to camera distance d from the origin to maintain visual clarity: intervals are configured as 0.5 for $d < 15$ units, 1.0 for $15 \leq d < 30$ units, 2.0 for $30 \leq d < 50$ units, 6.0 for $50 \leq d < 80$ units (default scale), and 12.0 for $d \geq 80$ units.

5.10 Minimap

A 2D circular minimap is rendered each frame on a `<canvas>` element overlaid on the viewport. It depicts all token positions projected onto the XZ plane, colored by the active color layer (Layer 1 by default), with a camera orientation indicator.

6. User Interface Design

6.1 Layout Philosophy

The interface adopts a three-panel editorial layout designed for scholarly rigor: a 280-pixel Left Sidebar hosting engine controls, model selection, and projection quality metrics; a central WebGL viewport for full-fidelity 3D exploration; and a 280-pixel Right Sidebar functioning as a topological inspector for the selected token. Typography is strictly structured using **EB Garamond** for editorial prose, **Fira Code** for monospaced metrics, and **MathJax 3** for mathematical expressions.

6.2 Left Sidebar: Control Dashboard

The control dashboard organizes administrative controls across five functional groups:

Model Selection. Interactive switching between Qwen 2.5 0.5B-Instruct, Gemma 2B, and BERT base uncased, with display of model architecture type (decoder-only / encoder-only), embedding dimensionality, and active vocabulary count N.

Manifold Properties. Displays active reduction algorithm, real-time animation FPS, and dataset hash.

Projection Engine. Provides interactive switching between UMAP, t-SNE, PCA (centered), PCA-NC (ablation), and Isomap algorithms, accompanied by spatial octant filters (Positive, Negative, or Full volume).

Visualization Layers. Toggle controls for the three color layers (Reference Partition / Thermal Camera / Neighborhood Probing) and the K value used for Layer 2 and Layer 3 visualization.

Camera Optics. Hosts projection mode selectors (Perspective, Parallel orthographic, Two-point perspective), field-of-view sliders ($10°$–$120°$), and direct preset triggers (Front, Top, Side, Isometric).

Cartesian Apparatus. Allows fine-grained toggling of coordinate axes (X, Y, Z), reference plane grids, mouse navigation sensitivity, and HUD indicators.

6.3 Right Sidebar: Token Inspector

Upon selecting a token via the central reticle, the inspector renders:

- The token's projected coordinate vector $\vec{v} = [x_{3d},\ y_{3d},\ z_{3d}]^\top$
- Global vocabulary matrix index and reference linguistic category badge
- Per-token behavioral indicator $B_{NX,i}(K)$ for the active K
- Ranked table of true neighbors (T_K), false neighbors (F_K), and missing neighbors (M_K) with cosine similarity scores
- Co-ranking curves ($M_T(K)$, $M_C(K)$, $B_{NX}(K)$, $Q_{NX}(K)$) for the current (model, algorithm) configuration
- Export trigger for downloading neighborhood data as CSV (`token_{label}_neighbors.csv`)

6.4 Comparative Metrics Panel (F5)

A dedicated panel displays the 12-configuration matrix ($3\ \text{models} \times 4\ \text{algorithms}$) with co-ranking metric curves over $K \in \{5, 10, 15, 20\}$. Each cell presents curves for $M_T(K)$, $M_C(K)$, $B_{NX}(K)$, $Q_{NX}(K)$, and the scalar Silhouette score. This panel enables direct cross-model and cross-algorithm comparison of projection fidelity.

6.5 Bottom Navigation HUD

A minimal floating overlay monitors real-time keypress states (W, A, S, D, R, F, ESC) with instantaneous typographic highlights, providing immediate visual feedback during flight navigation.

7. Data Model

7.1 TypeScript Interfaces

The frontend data model is structured around a set of TypeScript interfaces defined in `src/types.ts` that enforce compile-time verification across coordinate payloads, metric scores, and model metadata (see Appendix D, Listing D.4).

The `SemanticNode` interface includes both `id` (sequential index within the active dataset) and `token_id` (vocabulary index in the model's full vocabulary), addressing the discrepancy between sequential node ordering and actual model vocabulary identifiers.

7.2 Static Fallback Dataset

`src/data/semanticSpace.ts` contains pre-computed semantic nodes with UMAP-projected coordinates derived from actual Qwen 2.5 0.5B-Instruct execution on the Penn Treebank dataset, using the congelated parameters of Section 4.5. This ensures a meaningful experience even without the Python backend.

8. Limitations and Future Work

8.1 Current Limitations

L1 — Context-free embeddings. ENSy extracts exclusively the static input embedding matrices of each model — lookup tables applied before any contextual attention layer. This is a deliberate methodological decision ensuring direct comparability across the three architectures (Section 2.1), not an accidental limitation. Contextualized representations (hidden states from intermediate transformer layers) would require decisions about layer depth, context, and pooling strategy that would compromise cross-architecture comparability.

L2 — Causal attribution. The three models differ in architecture, embedding dimension, vocabulary, tokenizer, pre-training objective, corpus, and training recipe simultaneously. Observed differences in distortion patterns cannot be attributed exclusively to any single factor. The experiment design permits observing cross-model variation; it does not isolate architectural causality.

L3 — Sample size and statistical variability. The dataset N is determined by the single-token intersection criterion applied to three different tokenizers. Depending on the resulting N, co-ranking indicators may exhibit higher variability than with larger samples. Results are reported with the actual N rather than inflated artificially.

L4 — Single-machine deployment. Production deployment would require containerization (Docker) and production ASGI server configuration.

L5 — Rendering at $N \gg 1000$. The current architecture is optimized for the expected N range (150–200 tokens). Scaling beyond ~ 2000 tokens would require batched similarity computation, instanced mesh rendering, and GPU-side culling.

8.2 Future Work

F1 — Contextual embeddings. Sentence-level hidden state extraction from intermediate transformer layers would enable visualization of how token representations shift across semantic contexts, extending beyond the static input embedding geometry characterized in this work.

F2 — Anisotropy analysis. Real-time characterization of anisotropy in the embedding space (Ethayarajh, 2019; Cai et al., 2021) would provide additional geometric context complementing the co-ranking framework.

F3 — Trajectory visualization. Rendering interpolation paths between projection algorithms as visible trajectories would reveal embedding stability across algorithmic transformations.

F4 — Multi-session persistence. Server-side result caching with database-backed storage would enable longitudinal analysis and cross-session comparison without recomputing projections.

9. Conclusion

This work presented **ENSy (Embedding Navigation System)**, an interactive computational environment for the 3D navigation and quantitative geometric analysis of neural language model embedding spaces. By coupling a FastAPI/PyTorch backend with a WebGL/Three.js rendering engine, the system fulfills its core design and architectural objectives: extracting architecture-agnostic input embeddings across distinct neural architectures (Qwen 2.5 0.5B, Gemma 2B, BERT base uncased), projecting them into navigable 3D Riemannian manifolds across linear (PCA) and non-linear (t-SNE, UMAP, Isomap) dimensionality reduction engines at 60 FPS, and grounding geometric quality assessment in the rank-based co-ranking framework of Lee & Verleysen (2009).

The methodological framework establishes a strictly reproducible experimental protocol across 12 configurations (3 model architectures $\times$ 4 projection algorithms), driven by a curated, SHA-256-versioned lexical dataset derived from the Penn Treebank and congelated hyperparameter tables. By formulating the per-token behavioral decomposition $B_{NX,i}(K)$ alongside global Trustworthiness, Continuity, and $Q_{NX}(K)$ metrics, the system equips researchers with complementary perceptual and quantitative instruments — namely, real-time screen-space reticle probing, a three-layer topological color hierarchy, and a local distortion thermal overlay.

ENSy bridges the gap between abstract mathematical quality criteria and interactive spatial exploration, delivering a foundational, browser-native instrument for explainable artificial intelligence (XAI). Following the full empirical execution of the 12 experimental configurations, future research directions will expand beyond static input embedding matrices toward layer-wise contextual representations, cross-session comparative persistence, and automated anisotropy diagnostics in deep language models.

Appendix A: File Structure Reference

```
manifold/
├── app.py                          FastAPI backend server
├── dataset_builder.py              Penn Treebank dataset pipeline (21-step)
├── metrics.py                      Co-ranking metric suite (Q, M_T, M_C, B_NX, Q_NX, Silhouette)
├── requirements.txt                Unified Python dependencies (backend + testing)
├── index.html                      HTML template; MathJax configuration
├── package.json                    Node.js package manifest
├── vite.config.ts                  Vite build configuration; API proxy
├── tsconfig.json                   TypeScript compiler options
└── src/
    ├── main.tsx                    React application entrypoint
    ├── App.tsx                     Root component; state orchestration
    ├── types.ts                    TypeScript data model interfaces
    ├── index.css                   Global CSS styles
    ├── components/
    │   ├── SpaceVisualizer.tsx     Three.js renderer; flight controls; color layers
    │   ├── SidebarLeft.tsx         Control dashboard; model/algorithm selectors
    │   ├── SidebarRight.tsx        Token inspector; co-ranking metrics; CSV export
    │   ├── MetricsPanel.tsx        12-configuration comparative metrics panel (F5)
    │   └── BottomHUD.tsx           Navigation indicator
    └── data/
        ├── dataset.json            Congelated lexical dataset (PTB-derived, versioned)
        └── semanticSpace.ts        Static fallback dataset
```

Appendix B: API Reference

`GET /api/projection/{model_id}/{engine_type}`

Path parameters: - `model_id` in {`qwen`, `gemma`, `bert`} - `engine_type` in {`umap`, `pca`, `pca_nc`, `tsne`, `isomap`}

Response schema:

```
{
  status: "STABLE" | "LOADING",
  meta: {
    model: string,
    architecture: "decoder-only" | "encoder-only",
    embedding_dim: number,
    total_tokens: number,
    dataset_hash: string,
    algorithm_params: object,
    library_versions: object
  },
  nodes: Array<{
    id: number,
    label: string,
    token_id: number,
    category: "content_noun" | "content_verb" | "function_word",
    coords: {
      x3d: number, y3d: number, z3d: number
    },
    bnx_local: { [K: number]: number },        // B_{NX,i}(K) for K in {5,10,15,20}
    true_neighbors:    Array<{ label: string, cosine_similarity: number }>,
    false_neighbors:   Array<{ label: string, cosine_similarity: number }>,
    missing_neighbors: Array<{ label: string, cosine_similarity: number }>
  }>,
  metrics: {
    trustworthiness: { [K: number]: number },
    continuity:      { [K: number]: number },
    b_nx:            { [K: number]: number },
    q_nx:            { [K: number]: number },
    silhouette:      number
  }
}
```

`POST /api/dataset`

Triggers execution of the Penn Treebank dataset pipeline (Section 4.2). Returns the resulting `dataset.json` content and SHA-256 hash.

Appendix C: Keyboard Controls Reference

Input Binding	Subsystem / Operation	Functional Description
Click Viewport	Input Management	Engages browser pointer lock mode (`requestPointerLock()`) to capture mouse orientation.
`W` / `S`	6-DOF Flight Camera	Translates the camera position forward or backward along the horizontal XZ world plane.
`A` / `D`	6-DOF Flight Camera	Strafes the camera position left or right perpendicular to the current gaze direction vector.
`R` / `F`	6-DOF Flight Camera	Translates the camera position vertically along the global Cartesian Y-axis (ascend / descend).
Mouse Motion	6-DOF Flight Camera	Adjusts camera orientation (pitch and yaw) with clamped pitch angle to eliminate gimbal lock.
Scroll Wheel	Optical Controls	Adjusts focal distance zoom (translates along the local Z-axis in perspective mode; scales orthographic frustum).
`T`	Visualization	Toggles the Thermal Camera layer (Layer 2, $B_{NX,i}(K)$ overlay).
`H`	User Interface	Toggles visibility of the bottom flight navigation HUD overlay.
`P`	Developer Diagnostics	Outputs point cloud memory metrics, raycasting statistics, and vector state to the browser console.
`Escape`	Input Management	Releases browser pointer lock and restores standard cursor interaction mode.

Appendix D: Implementation Listings

Listing D.1 — Embedding Extraction and Normalization (`app.py`)

```
# Decoder-only models (Qwen, Gemma)
embedding_matrix = model.get_input_embeddings().weight.detach().float().numpy()

# Encoder-only models (BERT)
embedding_matrix = model.bert.embeddings.word_embeddings.weight.detach().float().numpy()

# L2 normalization (mandatory, applied to all models)
norms = np.linalg.norm(embedding_matrix, axis=1, keepdims=True)
embedding_matrix = embedding_matrix / norms
```

Listing D.2 — Asynchronous Model Loading (`app.py`)

```
@app.on_event("startup")
def startup_event():
    for model_id in ["qwen", "gemma", "bert"]:
        threading.Thread(
            target=load_model_background,
            args=(model_id,),
            daemon=True
        ).start()
```

Listing D.3 — Node Schema Serialization (`app.py`)

```
{
  "id": 0,
  "label": "document",
  "token_id": 6062,
  "category": "content_noun",
  "coords": {
    "x3d": 5.423237,
    "y3d": 2.238469,
    "z3d": 6.157989
  },
  "bnx_local": { "5": 0.12, "10": 0.08, "15": 0.05, "20": 0.03 },
  "true_neighbors":    [{ "label": "file",    "cosine_similarity": 0.8821 }],
  "false_neighbors":   [{ "label": "compile", "cosine_similarity": 0.4103 }],
  "missing_neighbors": [{ "label": "record",  "cosine_similarity": 0.7654 }]
}
```

Listing D.4 — TypeScript Data Model Interfaces (`src/types.ts`)

```
type LinguisticCategory = "content_noun" | "content_verb" | "function_word";
type KValue = 5 | 10 | 15 | 20;

interface Coords3D {
  x3d: number;
  y3d: number;
  z3d: number;
}

interface Neighbor {
  label: string;
  cosine_similarity: number;
}

interface SemanticNode {
  id?: number;
  token_id: number;
  label: string;
  category: LinguisticCategory;
  coords: Coords3D;
  bnx_local: Record<KValue, number>;
  true_neighbors:    Neighbor[];
  false_neighbors:   Neighbor[];
  missing_neighbors: Neighbor[];
}

interface CoRankingMetrics {
  trustworthiness: Record<KValue, number>;
  continuity:      Record<KValue, number>;
  b_nx:            Record<KValue, number>;
  q_nx:            Record<KValue, number>;
  silhouette:      number;
}

interface SemanticSpace {
  meta: SemanticMeta;
  nodes: SemanticNode[];
  metrics: CoRankingMetrics;
}
```

References

CAI, Xingyu et al. Isotropy in the Contextual Embedding Space: Clusters and Manifolds. **ICLR**, 2021.

ETHAYARAJH, Kawin. How Contextual are Contextualized Word Representations? **EMNLP-IJCNLP**, 2019.

LEE, John A.; VERLEYSEN, Michel. Quality assessment of dimensionality reduction: Rank-based criteria. **Neurocomputing**, v. 72, p. 1431–1443, 2009.

LUEKS, W. et al. How to Evaluate Dimensionality Reduction? Improving the Co-ranking Matrix. **arXiv:1110.3917**, 2011.

MCINNES, Leland et al. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. **arXiv**, 2018.

PEARSON, Karl. On lines and planes of closest fit. **Philosophical Magazine**, 1901.

ROUSSEEUW, Peter J. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. **J. Comput. Appl. Math.**, 1987.

TENENBAUM, Joshua B. et al. A global geometric framework for nonlinear dimensionality reduction. **Science**, 2000.

VAN DER MAATEN, Laurens; HINTON, Geoffrey. Visualizing data using t-SNE. **JMLR**, 2008.

VENNA, Jarkko; KASKI, Samuel. Local multidimensional scaling. **Neural Networks**, 2006.